Ioannis Papadopoulos AEM: 03899

Contents

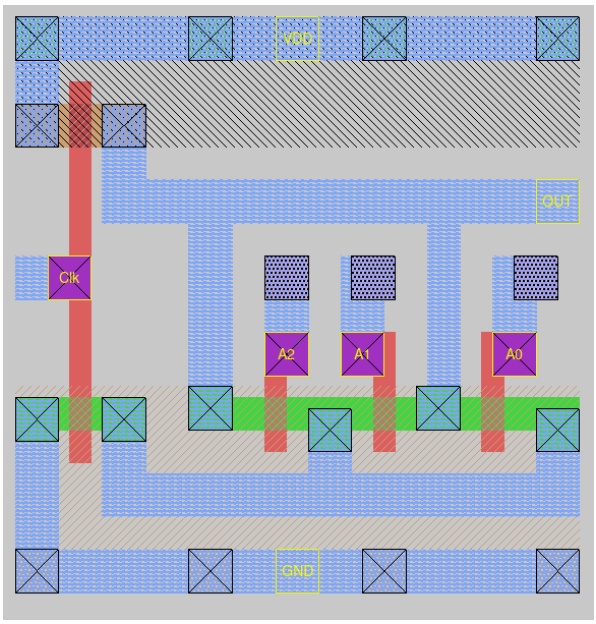
1. Part A – Decoder 2
1.1 Decoder Layouts 2
1.2 Decoder Verifications 4
2. Part B – 6T SRAM Cell 5
2.1 SRAM Layout 5
2.2 SRAM Verifications 5
3. Part C – Full Layout Implementation of an 8-SRAM Column. 7
3.1 8-SRAM Layout with NOR Logic Decoder 7
3.2 8-SRAM Layout with NAND Logic Decoder 9

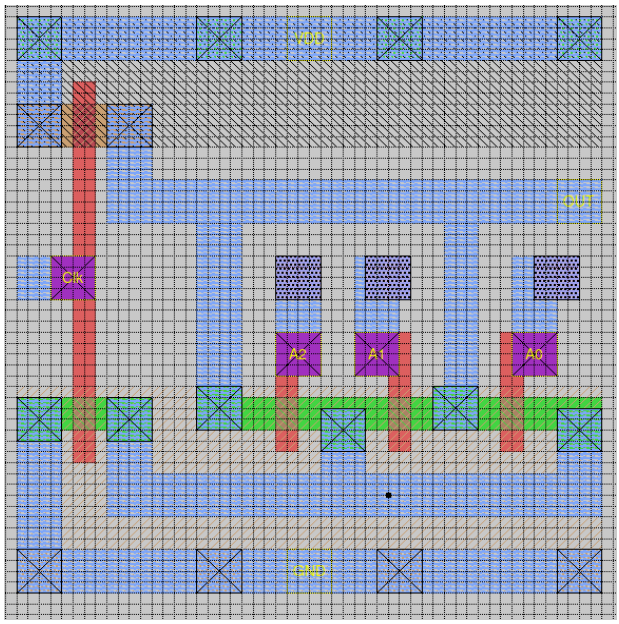
1. Part A – Decoder

In Part A, multiple circuit implementations were explored, all based on dynamic logic. The first two designs use NOR logic, while a third alternative uses NAND logic.

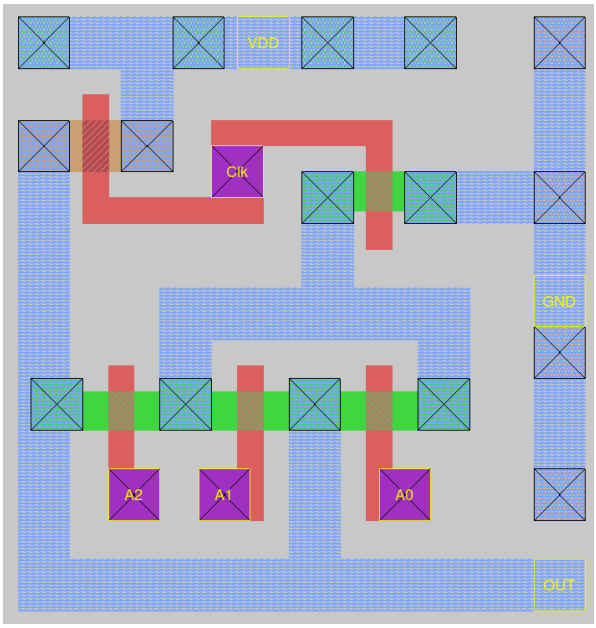
1.1 Decoder Layouts

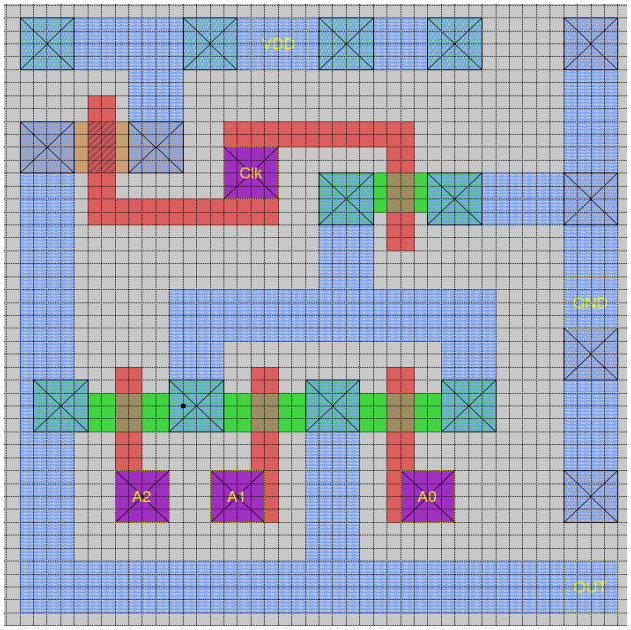
1st NOR design "Simple". Size = 52 x 53:





2nd NOR design. Size 42 x 56:

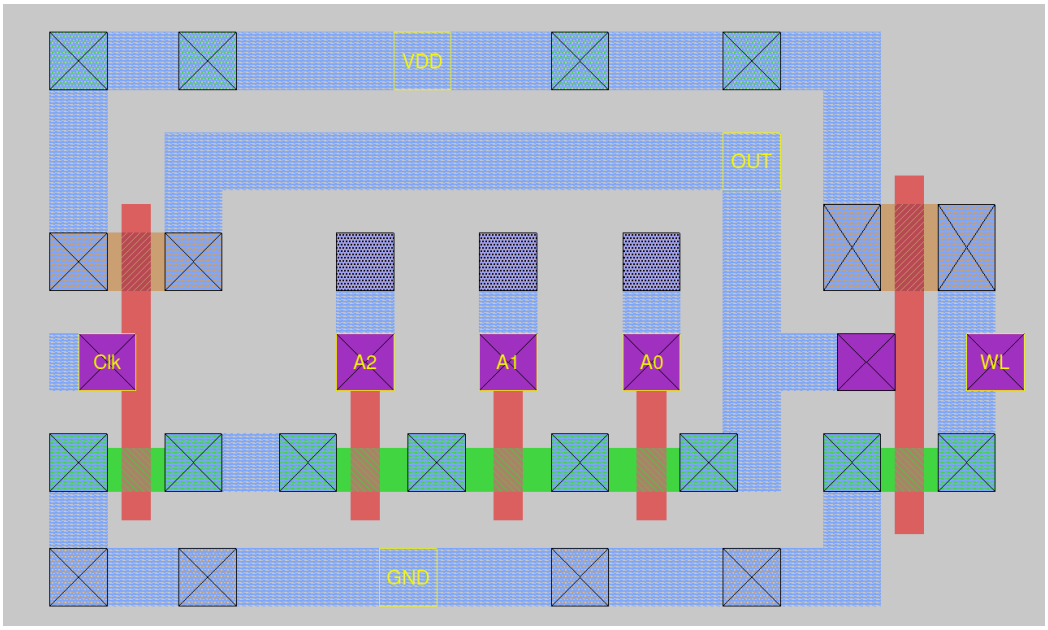


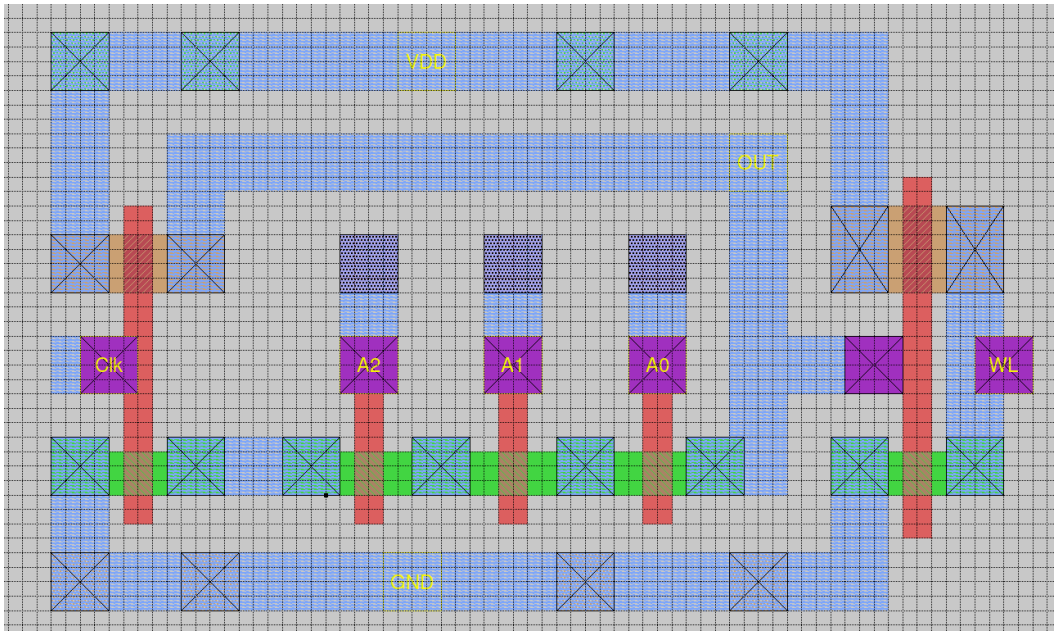


Between the two NOR-based designs, the first one was selected for further use. Although it occupies a larger area than the second design, it offers significantly better compatibility when integrating into the complete circuit in PartC. The second NOR design is more compact, but its ground rail is not located at the bottom of the layout, which is not usually advised.

The third design based on NAND logic was developed for reasons that will be discussed in PartC. But notice that is also smaller than the 1st NOR design (68x40 < 52x53).

NAND design. Size 60 x 40:

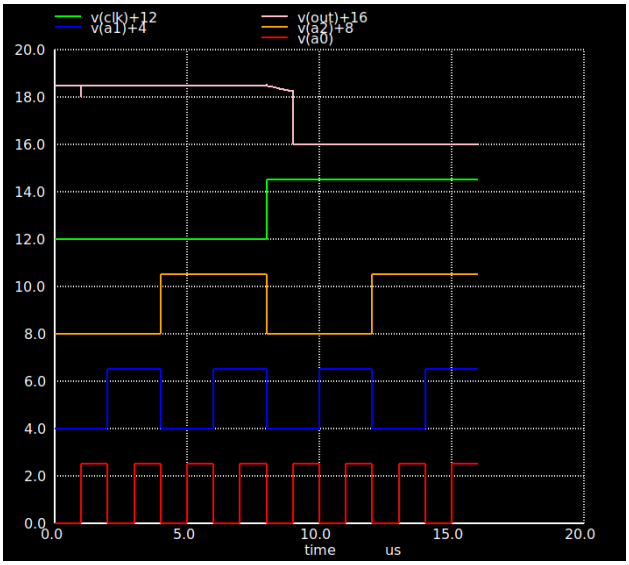




1.2 Decoder Verifications

All the designs were extracted to ngspice for verification. The 2 NOR designs have the same function, therefore only one waveform will be shown here.

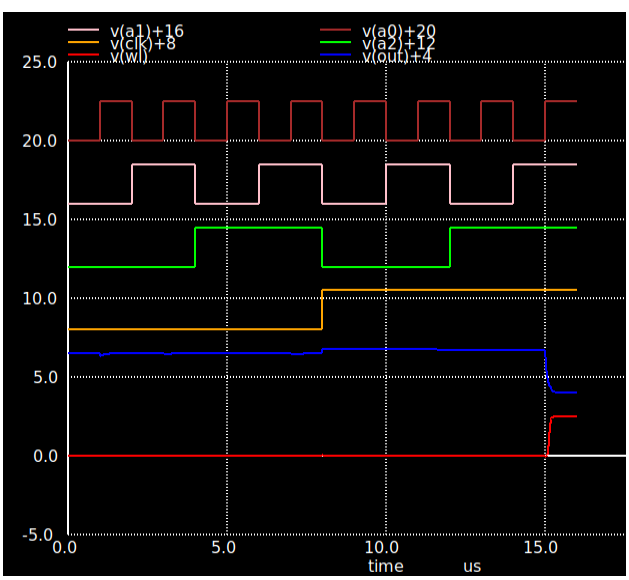
NOR waveform



Truth Table of 3-input NOR Gate			
A	B	C	Q
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

When the Clk is “0” then the OUT is correctly always high, but when the Clk changes to “1” then the PDN can change the OUT value. In this example A2, A1 and A0 are at “0” right after the Clk changes to “1”, so the OUT still stays at “1”. When the combination of the gates changes to 0 0 1, then the OUT drops to “0” and stays there.

NAND waveform



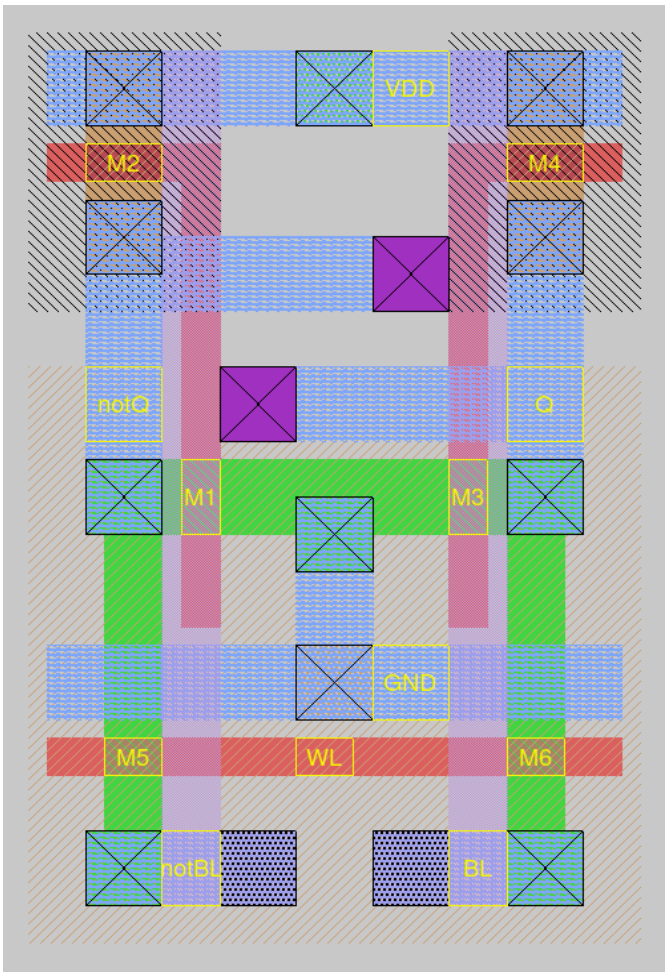
Truth Table of 3-input NAND Gate			
A	B	C	Q
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

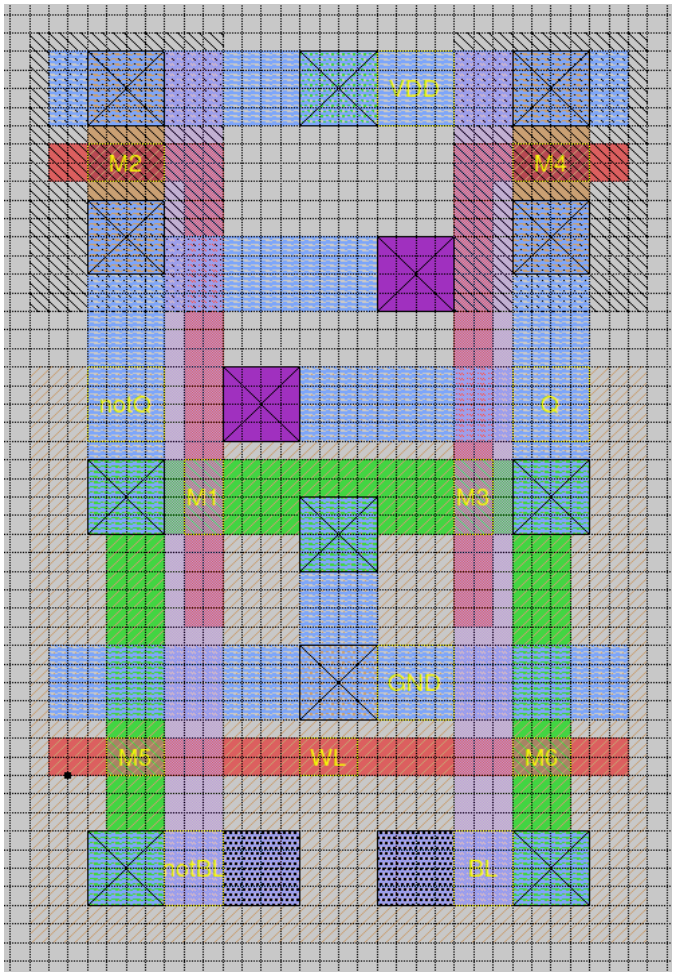
The same logic applies here, only after the Clk goes to “1” the PDN can drop the OUT to zero. Here this happens when A2, A1 and A0 are high. And the WL is always opposite of the OUT (more explanation later).

2. Part B – 6T SRAM Cell

2.1 SRAM Layout

Size 32x46



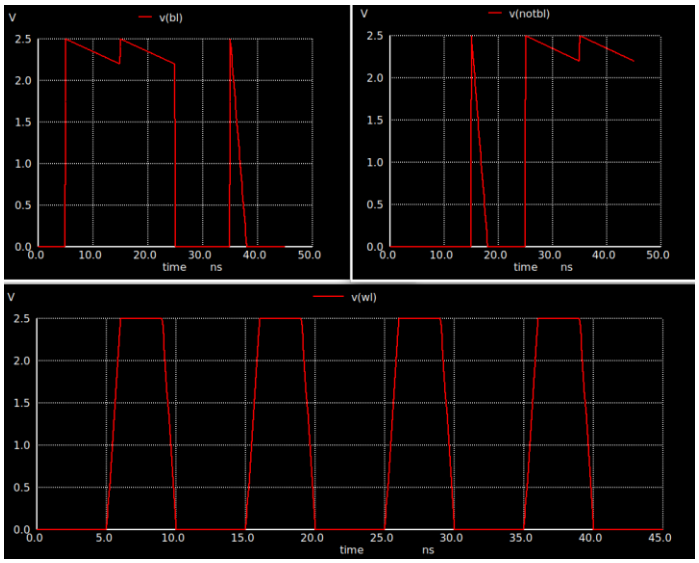


2.2 SRAM Verifications

To write and read in the SRAM some things must be prepared before the WL is set high.

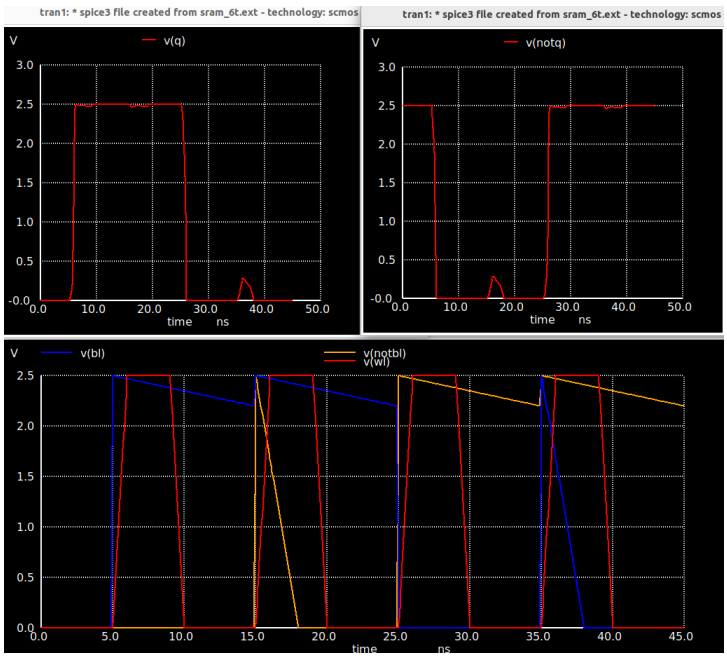
WRITE	READ
1. Charge BL high 2. Let BL float 3. Connect notBL to the ground 4. Set the WL high This way Q = “1” meaning that “1” was written in the SRAM. For the opposite result (Q= “0”), swap the BL with notBL in every step.	1. Charge BL and notBL high 2. Let BL and notBL float 3. Set the WL high

The test sequence will consist of four basic operations: write ‘1’, read ‘1’, write ‘0’, and read ‘0’.



As described in the previous table to **write “1”**, the BL is charged high and notBL is 0. Then the WL rises to 1 while the BL floats. Then after the WL drops to 0 again, the **read “1”** operation is being prepared. Both BL and notBL are charged high, then WL rises to “1” while the BL and not BL float. The **next 2 operations** are just the mirrored versions of the first 2 operations.

In the waveform below, the values of Q and notQ match the sequence. Q is “1” for the first 2 operations, and notQ is “1” for the next 2. Also, while reading the content inside the SRAM stays intact. There is a little bump for a bit, but that was expected and doesn’t cause any problem because the SRAM was designed with a cell ratio of CR = 4/3 > 1.2 and a pull-up ratio of PR = 4/3 < 1.8.

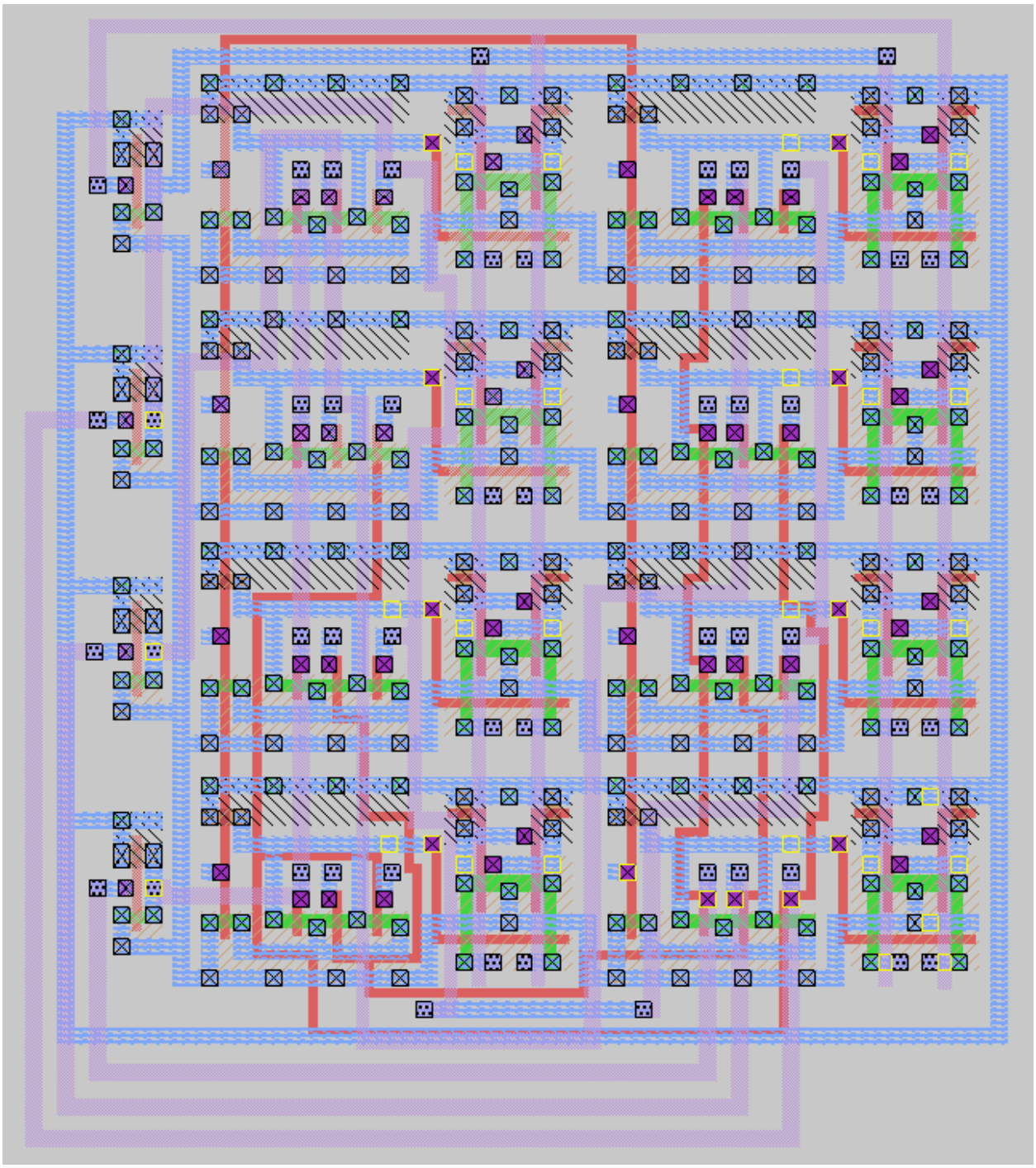


NOTE: In reality the floating part would not be linear, but I was using the PWL function. This doesn’t affect the result though.

3. Part C – Full Layout Implementation of an 8-SRAM Column.

In Part C, two complete layout implementations were developed using the first NOR-based decoder design and the NAND-based decoder design. The layouts were organized in a 4×2 configuration, where the decoder–SRAM pair is replicated eight times and arranged in two columns instead of a single long column. This approach achieves a more compact overall layout.

3.1 8-SRAM Layout with NOR Logic Decoder

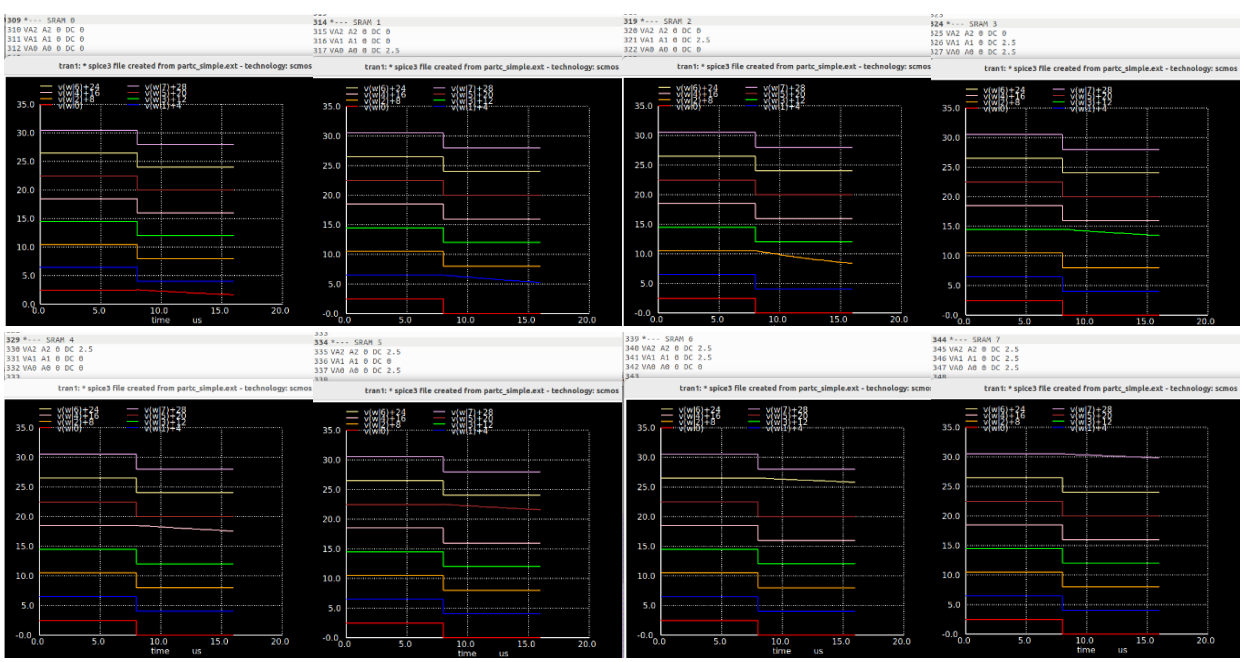


The overall layout occupies an area of 246 × 287 uλ² = 70,602 μλ². In the presented layout, the bottom-right decoder generates the WL0 corresponding to the sequence “000”, while the upper-left decoder generates WL7 corresponding to the sequence “111”. The remaining word lines are arranged sequentially between these two extremes, with WL1 located above WL0 and WL6 located below WL7, following the natural address order. Three inverters are placed on the left of the layout. Starting from the top, is the inverter for the A0, next is the inverter for notA1 and lastly for notA2. (Ignore the very top inverter, it is removed later in the NAND layout. It’s a mistake)

Verification

Each waveform displays all eight WLs, and above them are three additional ngspice signals corresponding to the input address bits A2, A1, and A0 used in each test case. Initially, all WLs remain high because the clock signal is set to 0, causing the pull-up network of each decoder to conduct. At approximately 8 μs, the clock transitions to 1, activating the evaluation phase. At this point, seven out of the eight WLs immediately discharge to 0, while only the word line corresponding to the applied A2–A1–A0 address combination remains high. This behavior confirms that the decoder operates correctly and exhibits the intended one-hot functionality.

The waveforms are easier to understand when looking back to “1.2 Decoder Verification” segment, in this report.

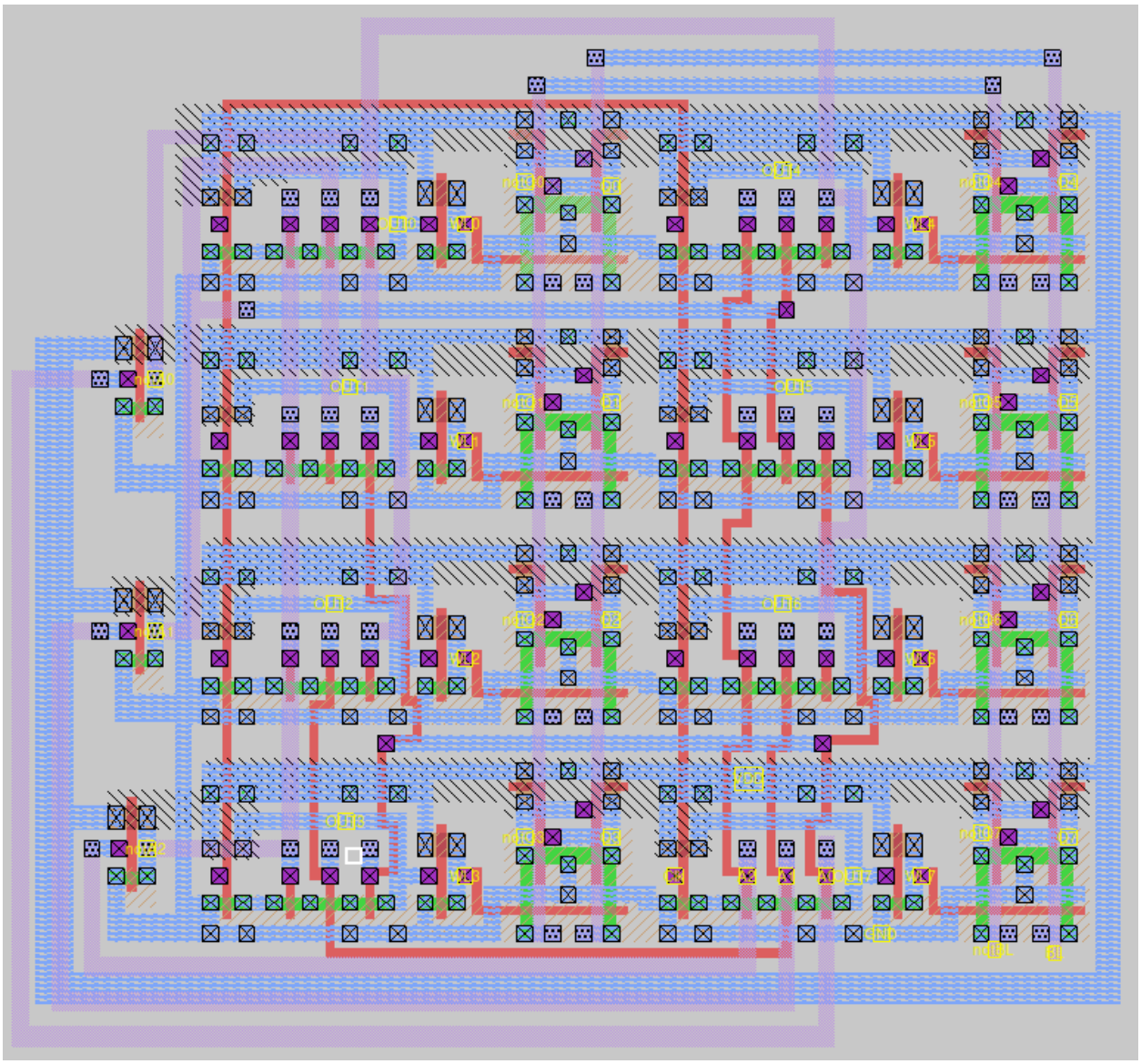


The word line that remains high after the clock transitions to logic ‘1’ is observed to slowly discharge, which is not desirable and indicates that the addition of a level restorer would stabilize the node. However, the main motivation for developing the NAND-based decoder was not this issue, but rather the fact that in the NOR-based dynamic design all word lines are high simultaneously during the precharge phase (CLK = 0). Although this condition exists only while the clock is low, it is not ideal for SRAM operation. Adding an inverter at the output of each NOR decoder would not solve this problem, since when CLK = 1 all non-selected decoders would rise

to logic ‘1’ while the selected one would fall to ‘0’, whereas an SRAM cell requires an active-high word line.

By switching to a NAND-based decoder, an inverter can be placed at the output of each decoder such that all word lines remain at logic ‘0’ during both the precharge and evaluation phases, except for the selected one. This is possible because the NAND truth table naturally produces a single logic ‘0’, in contrast to NOR logic which produces a single logic ‘1’. As a result, the NAND-based design always guarantees true one-hot word-line activation, independent of the clock phase. For this reason, the NOR-based design was not further improved with a level restorer, since the NAND-based approach was ultimately preferred. Nevertheless, if the condition of all word lines being high during CLK = 0 is not considered problematic, the NOR-based decoder remains functionally correct, if memory operations are performed only during the evaluation phase (CLK = 1).

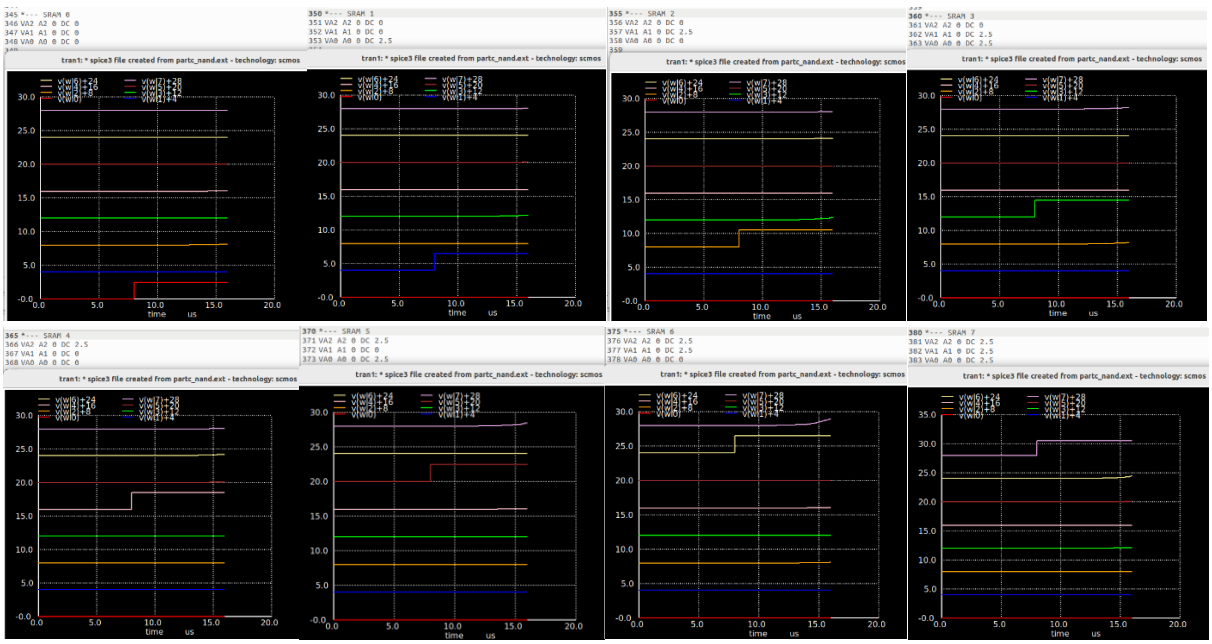
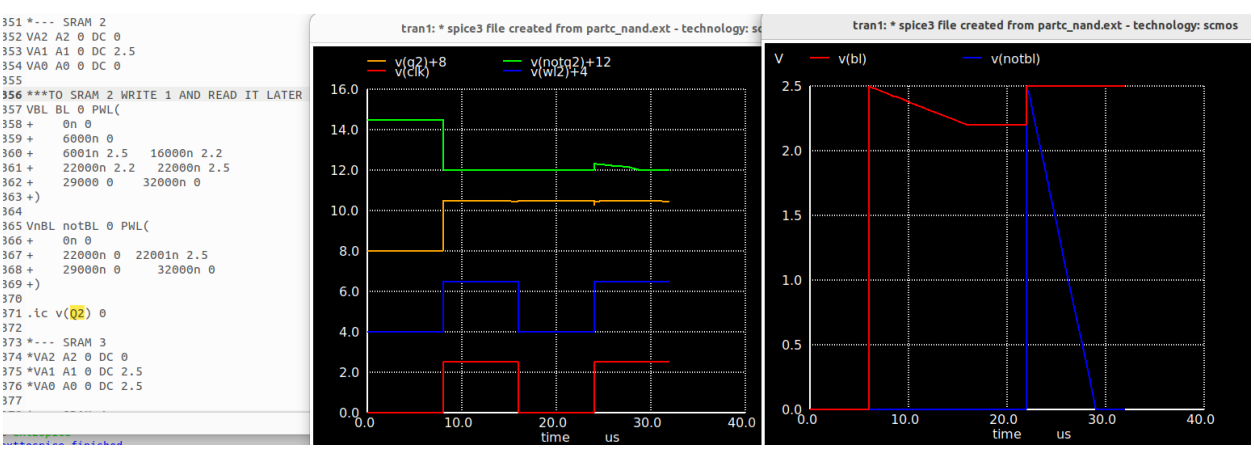
3.2 8-SRAM Layout with NAND Logic Decoder



This time the overall layout occupies an area of 279 × 258 uλ² = 71,982 μλ². Now in the presented layout, the bottom-right decoder generates the WL7 still corresponding to the sequence “111”, while the upper-left decoder generates WL0 still corresponding to the sequence “000”. Also, the WL1 is located below WL0 and WL6 is located above WL7, and so on. Four inverters are placed on the left of the layout. Starting from the top, is the inverter for the A0, next is the inverter for notA1 and lastly for notA2. I want to also mention that this design came out way cleaner than the previous one, therefore it is probably easier to design physically.

Verification

The verification is carried with the same idea as explained before, only this time because of the inverter and the NAND logic, when clock is 0 the WL is 0 and when clock is 1, only the corresponding WL goes to 1. Also this time there is no need for any extra circuit like a level restorer



Below is a test that activates only the WL2 and writes 1 and later or reads it. The Q2 is initialized at 0, thus the notQ2 to 1. When Clk changes to 1, the writing begins so Q2 = 1, notQ2 = 0.